

LEVEL 4 — DEVELOPMENT TRACK FOUNDATIONS

APIs & Data Engineering

The plumbing that gets external data into your systems reliably

Covers: REST/WebSockets · authentication · rate limits · retry logic · streaming market data · data ingestion/ETL · data validation/versioning/lineage · historical storage · real-time pipelines

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

Table of Contents	2
13. APIs & Data Engineering (Level 4)	3
13.1 Learning Position at a Glance	3
13.2 REST & WebSockets	3
13.3 Authentication	4
13.4 Rate Limits	5
13.5 Retry Logic	5
13.6 Streaming Market Data	6
13.7 Data Ingestion & ETL	7
13.8 Data Validation, Versioning & Lineage	8
13.9 Historical Storage & Real-Time Pipelines	8
13.10 Worked Exercise — Model Answer	8
13.11 Common Mistakes — Deeper Look	9
13.12 Advanced Extensions	10
13.13 Professional Applications	11

13. APIs & Data Engineering (Level 4)

Covers: REST/WebSockets · authentication · rate limits · retry logic · streaming market data · data ingestion/ETL · data validation/versioning/lineage · historical storage · real-time pipelines

13.1 Learning Position at a Glance

- **What it is:** The plumbing that gets external data (market data, broker connectivity) into your systems reliably.
- **Why it matters:** Nearly every real quant system depends on external data feeds; unreliable ingestion (dropped messages, silent schema changes) silently corrupts downstream research and trading.
- **Prerequisites:** Software Engineering Foundations; Databases.
- **Connects to Research:** Data quality issues upstream (bad ticks, missing bars, corporate-action errors) are one of the most common causes of false research findings.
- **Connects to Development:** This is core Level 4 subject matter — building robust ingestion and ETL pipelines.
- **Connects to Trading:** A dropped WebSocket connection to a live market-data feed with no reconnect/retry logic is a direct trading-risk event.
- **Practical exercise:** Build a WebSocket client that ingests a live (or simulated) market-data stream, validates each message against an expected schema, and handles disconnects with exponential backoff retry.
- **Common beginner mistakes:**
 - No data validation on ingestion, allowing corrupted or stale data to silently propagate downstream.
 - No retry/backoff logic on API calls, causing brittle failures on transient network issues.
- **Advanced extensions:** Real-time streaming architectures (e.g. message queues) for multi-consumer data pipelines.
- **Professional applications:** Market-data ingestion; broker connectivity; feature-store population.

13.2 REST & WebSockets

REST is a request-response style: the client asks a specific question ('what is the latest price for AAPL?') and the server answers once, over a connection that is then closed. A WebSocket is a persistent, two-way connection: once established, the server can push new messages to the client the instant they occur, with no need for the client to ask repeatedly.

REST polling vs. WebSocket streaming for market data

REST polling: client asks repeatedly, most requests return 'nothing new'



WebSocket: one persistent connection, server pushes the instant an event occurs

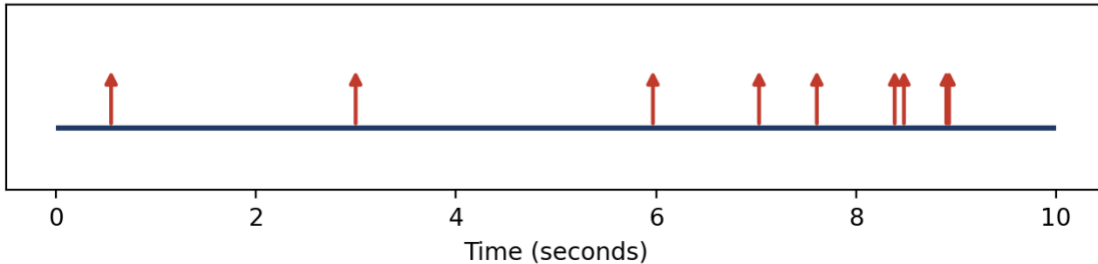


Figure 13.1 — REST polling repeats a request on a fixed schedule, wasting requests when nothing has changed and adding latency up to one full polling interval; a WebSocket delivers each update the moment it happens, over a single long-lived connection.

Example 13.1 — A simple REST request for a single quote

```
import requests

# REST: a single request-response round trip
response = requests.get(
    "https://api.example.com/v1/quote",
    params={"symbol": "AAPL"},
    headers={"Authorization": f"Bearer {api_key}"},
)
quote = response.json()
```

- **Use REST for:** Infrequent, on-demand lookups — reference data, historical bars, account balances — where a single answer at a single point in time is what's needed.
- **Use WebSockets for:** Continuous, real-time data where latency matters and updates arrive frequently and unpredictably — live quotes, order book updates, trade prints — exactly the shape of data the practical exercise's live market-data stream has.

13.3 Authentication

Authentication proves who is making a request, so that a data provider or broker can grant (or deny) access appropriately and attribute usage correctly. Credentials must be treated as sensitive: never hardcoded into source code (which would put them directly into Git history, per §10.2) and never logged in plaintext (per §9.7).

Example 13.2 — Loading credentials from the environment, never from source code

```
import os

# Load credentials from environment variables, never hardcoded
API_KEY = os.environ["MARKET_DATA_API_KEY"]
API_SECRET = os.environ["MARKET_DATA_API_SECRET"]

headers = {"Authorization": f"Bearer {API_KEY}"}
```

```
# .gitignore (see Example 10.2) must exclude any file holding these,  
# such as a local .env file used to set the environment variables above.
```

- **API keys:** A simple, static secret identifying the caller; easy to use but must be rotated if ever exposed.
- **OAuth tokens:** Short-lived, revocable tokens obtained through an authentication flow; more complex to implement but limit the damage if a token leaks, since it naturally expires.
- **Signed requests:** Some broker APIs require signing each request with a private key rather than sending a shared secret directly, so the secret itself is never transmitted at all.

13.4 Rate Limits

A rate limit caps how many requests a client may make in a given time window, protecting the provider's infrastructure from being overwhelmed. Exceeding a rate limit typically results in requests being rejected (often with an HTTP 429 'Too Many Requests' response) until the window resets; a well-behaved client tracks its own usage and paces requests to stay under the limit, rather than discovering the limit only by hitting it repeatedly.

Example 13.3 — A simple client-side rate limiter that paces requests proactively

```
import time  
from collections import deque  
  
class RateLimiter:  
    """Allows at most `max_calls` calls per `period` seconds."""  
    def __init__(self, max_calls: int, period: float):  
        self.max_calls = max_calls  
        self.period = period  
        self.calls = deque()  
  
    def wait_if_needed(self) -> None:  
        now = time.monotonic()  
        while self.calls and now - self.calls[0] > self.period:  
            self.calls.popleft()  
        if len(self.calls) >= self.max_calls:  
            time.sleep(self.period - (now - self.calls[0]))  
        self.calls.append(time.monotonic())
```

Respecting rate limits proactively, rather than reactively handling rejections, keeps a data pipeline's throughput smooth and predictable and avoids the provider imposing a harsher, longer-lived penalty (or an outright ban) for repeated violations.

13.5 Retry Logic

Network calls fail for reasons entirely unrelated to the request itself — a transient DNS hiccup, a momentary server overload, a dropped connection — and immediately giving up on the first such failure makes a pipeline needlessly fragile. Retrying with exponential backoff — waiting progressively longer between each retry attempt — gives a transient problem time to resolve itself while avoiding hammering an already-struggling server with an immediate flood of retries.

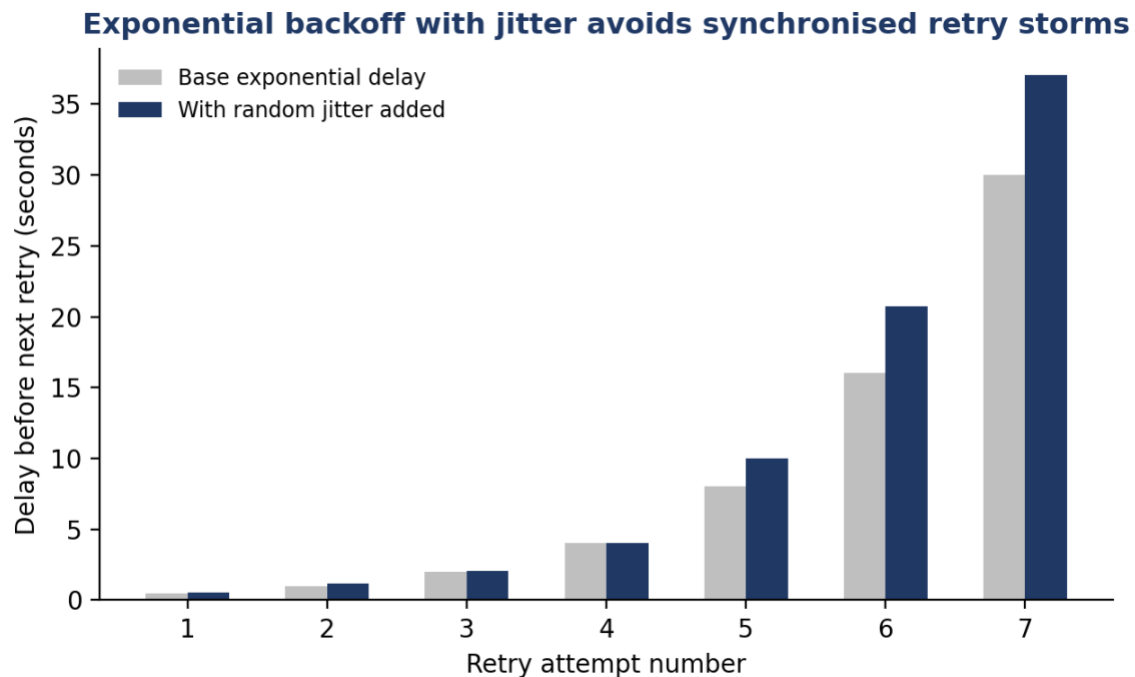


Figure 13.2 — Each retry attempt waits roughly twice as long as the last (capped at a maximum), with a small random jitter added so that many clients retrying the same failure do not all retry at exactly the same instant and re-create the overload they are backing off from.

Example 13.4 — Exponential backoff with jitter around a flaky network call

```
import time, random
import requests

def fetch_with_retry(url: str, max_attempts: int = 6, base_delay: float = 0.5):
    for attempt in range(1, max_attempts + 1):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()
            return response.json()
        except (requests.ConnectionError, requests.Timeout, requests.HTTPError) as exc:
            if attempt == max_attempts:
                raise
            delay = min(base_delay * 2 ** (attempt - 1), 30)
            delay += random.uniform(0, delay * 0.3) # jitter
            logger.warning("Attempt %d failed (%s); retrying in %.1fs",
                          attempt, exc, delay)
            time.sleep(delay)
```

- **Retry transient errors only:** Connection errors, timeouts, and 5xx server errors are typically worth retrying; a 4xx client error (bad request, invalid credentials) will fail identically on every retry and should not be retried blindly.
- **Always cap the number of attempts:** An unbounded retry loop against a genuinely broken endpoint will retry forever; a maximum attempt count, after which the failure is raised and handled explicitly, is essential.

13.6 Streaming Market Data

A streaming market-data client must handle a fundamentally different failure mode from a REST client: rather than a single request that either succeeds or fails, a WebSocket connection can drop unexpectedly mid-stream at any time, and every message that would have arrived during the disconnect is simply lost unless the client reconnects and, where the provider supports it, catches up on missed messages.

Example 13.5 — A reconnecting WebSocket client with exponential backoff on disconnect

```

import asyncio
import websockets

async def stream_market_data(url: str, on_message):
    attempt = 0
    while True:
        try:
            async with websockets.connect(url) as ws:
                attempt = 0 # reset backoff after a successful connection
                logger.info("Connected to market data stream")
                async for raw_message in ws:
                    on_message(raw_message)
        except (websockets.ConnectionClosed, OSError) as exc:
            attempt += 1
            delay = min(0.5 * 2 ** (attempt - 1), 30)
            logger.warning("Stream disconnected (%s); reconnecting in %.1fs",
                           exc, delay)
            await asyncio.sleep(delay)

```

Note the attempt counter is reset to zero on every successful connection: backoff should escalate for a run of consecutive failures, not accumulate indefinitely across an otherwise healthy connection's occasional, unrelated disconnects.

13.7 Data Ingestion & ETL

ETL — Extract, Transform, Load — describes the standard shape of a data pipeline: pull raw data from a source, reshape it into the form the target system expects, and write it to storage. Inserting an explicit Validate step between Extract and Transform (rather than only checking data quality much later, if at all) means bad data is caught at the earliest possible point, before it has a chance to contaminate any downstream transformation or storage.

An ETL pipeline with an explicit failure path, not a silent one

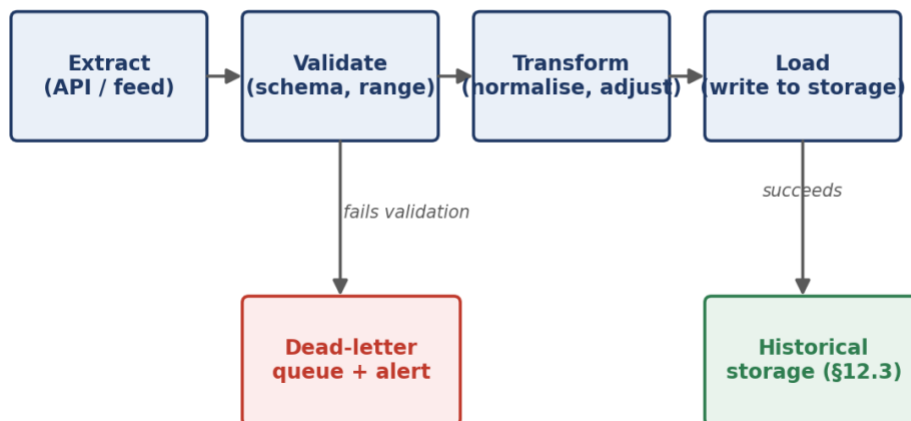


Figure 13.3 — A pipeline with an explicit dead-letter queue: data failing validation is routed somewhere visible and alertable, rather than being silently dropped or, worse, silently passed through to storage anyway.

The dead-letter queue in Figure 13.3 is a deliberate design choice, not an afterthought: without it, a validation failure has only two options — crash the entire pipeline (halting ingestion of every other, valid message) or silently discard the bad message (the exact common mistake in §13.11). Routing failures to a separate, monitored queue preserves both continuity (other messages keep flowing) and visibility (a human is alerted to investigate).

13.8 Data Validation, Versioning & Lineage

This builds directly on the data-quality discipline from §6.4 and the automated validation checks from §11.7, applied specifically at the point data enters a pipeline from an external source — exactly where a provider's silent schema change or data-quality regression first becomes visible.

- **Validation:** Checking every ingested message against an expected schema and plausible value ranges before accepting it, exactly as the practical exercise requires for the WebSocket client.
- **Versioning:** Recording which version of a dataset (or which provider API version) produced a given piece of stored data, so a later provider change can be distinguished from a genuine change in the underlying market — directly extending the reproducibility discipline from §6.5 and §10.6 to external data as well as code.
- **Lineage:** Tracking the full chain from raw source through every transformation to final stored (or consumed) form, so that when a downstream number looks wrong, its entire provenance can be traced back to find exactly where the problem was introduced.

Example 13.6 — Schema validation for each incoming streaming message

```
import jsonschema

TICK_SCHEMA = {
    "type": "object",
    "required": ["symbol", "price", "size", "timestamp"],
    "properties": {
        "symbol": {"type": "string", "minLength": 1},
        "price": {"type": "number", "exclusiveMinimum": 0},
        "size": {"type": "number", "minimum": 0},
        "timestamp": {"type": "number"},
    },
}

def validate_tick(message: dict) -> None:
    jsonschema.validate(instance=message, schema=TICK_SCHEMA)
```

13.9 Historical Storage & Real-Time Pipelines

Once validated, streaming data typically needs to serve two different consumers with very different latency requirements: a real-time pipeline that acts on data within milliseconds (a live strategy, a risk monitor), and historical storage (§12.3, §12.6) that persists the same data for later research and backtesting. Designing for both from the start — rather than bolting historical storage on as an afterthought — avoids the two paths silently drifting into inconsistent views of the same underlying data.

- **Real-time path:** Optimised for latency: minimal processing between message receipt and the consumer acting on it, typically in-memory, with persistence happening asynchronously alongside rather than blocking the real-time path.
- **Historical path:** Optimised for completeness and queryability: every validated message durably written to the schema and indexing structure from §12.3–§12.5, supporting the multi-year backtests §6.1–§6.7 depend on.

13.10 Worked Exercise — Model Answer

Exercise: Build a WebSocket client that ingests a live (or simulated) market-data stream, validates each message against an expected schema, and handles disconnects with exponential backoff retry.

Model answer — a validated, auto-reconnecting market-data client

```
import asyncio, json, logging, random
import websockets
import jsonschema

logger = logging.getLogger("market_data_client")
```

```

TICK_SCHEMA = {
    "type": "object",
    "required": ["symbol", "price", "size", "timestamp"],
    "properties": {
        "symbol": {"type": "string", "minLength": 1},
        "price": {"type": "number", "exclusiveMinimum": 0},
        "size": {"type": "number", "minimum": 0},
        "timestamp": {"type": "number"},
    },
}

class MarketDataClient:
    def __init__(self, url: str, on_tick):
        self.url = url
        self.on_tick = on_tick
        self._attempt = 0

    async def run(self) -> None:
        while True:
            try:
                async with websockets.connect(self.url) as ws:
                    self._attempt = 0
                    logger.info("Connected to %s", self.url)
                    async for raw in ws:
                        self._handle_message(raw)
            except (websockets.ConnectionClosed, OSError) as exc:
                await self._backoff(exc)

    def _handle_message(self, raw: str) -> None:
        try:
            message = json.loads(raw)
            jsonschema.validate(instance=message, schema=TICK_SCHEMA)
        except (json.JSONDecodeError, jsonschema.ValidationError) as exc:
            logger.warning("Dropping invalid message: %s (%s)", raw, exc)
            return # routed to a dead-letter log rather than crashing
        self.on_tick(message)

    async def _backoff(self, exc: Exception) -> None:
        self._attempt += 1
        delay = min(0.5 * 2 ** (self._attempt - 1), 30)
        delay += random.uniform(0, delay * 0.3)
        logger.warning("Disconnected (%s); reconnecting in %.1fs", exc, delay)
        await asyncio.sleep(delay)

```

This directly combines three practices from this section: streaming connection handling with automatic reconnection (§13.6), schema validation on every single message before it reaches any downstream consumer (§13.8), and exponential backoff with jitter specifically on the reconnect path rather than on individual messages (§13.5) — exactly the three requirements the exercise names.

13.11 Common Mistakes — Deeper Look

Mistake: No data validation on ingestion, allowing corrupted or stale data to silently propagate downstream

An external data provider is a black box whose behaviour can change without notice — a field silently renamed, a unit silently changed from cents to dollars, a feed that quietly stops updating while technically remaining 'connected.' Without validation at the point of ingestion, none of these changes announce themselves; they simply become slightly-wrong data flowing into research (feeding directly into the false-discovery risk from §6.7 and §11.6) or, worse, into a

live trading decision. The schema and range checks in §13.8 and Example 13.6 exist specifically to convert an invisible, silent problem into a loud, immediate, actionable one.

Mistake: No retry/backoff logic on API calls, causing brittle failures on transient network issues

A pipeline that treats every network hiccup as a fatal, unrecoverable error is far more fragile than the underlying infrastructure actually is: most real-world network failures are transient and resolve within seconds, and a client with no retry logic fails an entire ingestion run over a problem that would have disappeared on its own. Conversely, retrying immediately and repeatedly without backoff (per §13.5) can turn a brief provider-side hiccup into a self-inflicted denial-of-service, making the underlying problem worse rather than better.

⚠ Case study: a silent unit change that corrupted weeks of research

- A data provider changed a volume field from shares to round lots (multiples of 100 shares) in a minor API update, with the change noted only in provider release notes that the ingestion pipeline's maintainers did not see.
- Because the ingestion pipeline performed no range validation on the volume field, the change passed through completely silently — values were still positive numbers of a plausible order of magnitude, just off by a factor of 100 for the affected instruments.
- Several weeks of volume-weighted research (execution cost estimates, liquidity-based universe filters) were quietly built on the corrupted data before a researcher noticed an unrelated result did not match a rough hand calculation; adding the range and cross-provider sanity checks from §13.8 afterward would have caught the change within the first hour it occurred.

13.12 Advanced Extensions

Real-time streaming architectures (e.g. message queues) for multi-consumer data pipelines: as more systems need the same live data — a trading strategy, a risk monitor, historical storage — having each one maintain its own separate WebSocket connection to the external provider duplicates connections unnecessarily and multiplies the provider-side load. A message queue (such as Kafka) sits between the single external connection and every internal consumer, decoupling them from each other and from the provider.

A message queue lets one data stream feed many independent consumers

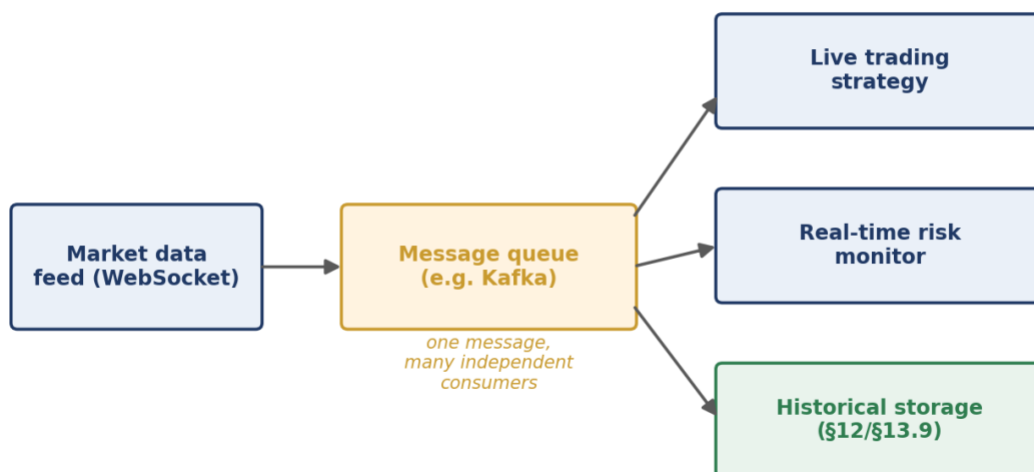


Figure 13.4 — A single ingestion client maintains the one external connection and publishes every validated message once to the queue; any number of internal consumers can subscribe independently, add or remove consumers without touching the ingestion client, and each consumer processes messages at its own pace.

- **Decoupling:** The ingestion client does not need to know what consumes its data, or how many consumers exist; consumers can be added or removed without changing the ingestion path at all.

- **Independent failure handling:** If the real-time risk monitor in Figure 13.4 crashes and restarts, the live trading strategy and historical storage consumers are entirely unaffected — each consumer's failures are isolated from the others.

13.13 Professional Applications

- **Market-data ingestion:** The full discipline in this section — authenticated, rate-limited, retrying REST and WebSocket clients, validated on ingestion, versioned and lineage-tracked — is the concrete implementation of reliable market-data infrastructure that every downstream research and trading system ultimately depends on.
- **Broker connectivity:** The same retry, backoff and validation discipline applies directly to order and execution connectivity, where a dropped connection or a malformed fill message carries direct financial risk, not just a research inconvenience.
- **Feature-store population:** A validated, versioned, lineage-tracked ingestion pipeline is what allows a feature store (§12.11) to be trusted as a single source of truth that both research and production systems can rely on consistently.